

1 Scope and program architecture

The supported gauge algebra is a finite semisimple algebra

$$\mathfrak{g} = \bigoplus_{a=1}^s \mathfrak{g}_a, \quad G = \prod_{a=1}^s G_a, \quad (1)$$

where every $\mathfrak{g}_a$ has Cartan type $A_r, B_r, C_r, D_r, E_6, E_7, E_8, F_4$, or G_2 . The compact group convention is the simply connected one used by the shared Lie-algebra module; in particular C_r is written $\mathrm{Sp}(r)$. Abelian gauge factors are outside the current input model.

The calculation has four layers:

1. `check_input_data` validates the existing simple- or product-group JSON convention and resolves names to Dynkin labels.
2. `_build_form_program` constructs the truncated plethystic exponential with formal character symbols.
3. `CharacterDecompositionCache` asks LiE for the necessary virtual character decompositions and caches decompositions and singlet coefficients in SQLite.
4. `_project_terms` extracts the gauge singlet factor by factor and `_to_sage_polynomial` packages the exact answer.

This division is important: FORM never needs to know a root system, while LiE never needs to manipulate the fugacities t, y, u .

2 Index convention and single-letter functions

Choose the supercharge defining the right index. With the conventions of Gadde, Pomoni, and Rastelli, the BPS condition and trace are [1]

$$\delta_R = \Delta - 2\bar{j} - 2R + r = 0, \quad (2)$$

$$\mathcal{I}(t, y, v) = \mathrm{Tr}_{\delta_R=0} (-1)^F t^{2(\Delta+\bar{j})} y^{2j} v^{-r-R}. \quad (3)$$

The code substitutes

$$v = u^2, \quad (4)$$

so every computational power of u is integral.

The vector and half-hypermultiplet single-letter indices are [1]

$$f_V(t, y, u) = \frac{t^2 u^2 - t^4 u^{-2} - t^3(y + y^{-1}) + 2t^6}{(1 - t^3 y)(1 - t^3 y^{-1})}, \quad (5)$$

$$f_H(t, y, u) = \frac{t^2 u^{-1} - t^4 u}{(1 - t^3 y)(1 - t^3 y^{-1})}. \quad (6)$$

The denominator counts the two compatible derivatives:

$$\frac{1}{(1 - t^3 y)(1 - t^3 y^{-1})} = \sum_{p, q \geq 0} t^{3(p+q)} y^{p-q}. \quad (7)$$

FORM uses a finite rectangular sum with $0 \leq p, q \leq \lfloor D/3 \rfloor$. Terms whose combined t degree exceeds D are removed automatically.

3 Matter characters for simple and product groups

For a product group, an irreducible external tensor product is

$$\mathbf{R}_h = \boxtimes_{a=1}^s R_{h,a}, \quad \chi_{\mathbf{R}_h}(g_1, \dots, g_s) = \prod_{a=1}^s \chi_{R_{h,a}}(g_a). \quad (8)$$

An omitted representation in the JSON input is the singlet, whose character is one.

The adjoint representation of the product algebra is a direct sum,

$$\mathfrak{ad}_{\mathfrak{g}} = \bigoplus_{a=1}^s (\mathbf{1}, \dots, \mathfrak{ad}_{\mathfrak{g}_a}, \dots, \mathbf{1}), \quad \chi_{\mathfrak{ad}_{\mathfrak{g}}} = \sum_{a=1}^s \chi_{\mathfrak{ad}_a}. \quad (9)$$

This explains why vector-multiplet characters are added, whereas a bifundamental hypermultiplet character is multiplied across its gauge factors.

For multiplicity N_h , the character measured in half-hyper units is

$$\hat{\chi}_h = \begin{cases} N_h \chi_{\mathbf{R}_h}, & \text{half hyper,} \\ N_h (\chi_{\mathbf{R}_h} + \chi_{\overline{\mathbf{R}}_h}), & \text{full hyper.} \end{cases} \quad (10)$$

Conjugation is simultaneous and factorwise:

$$\overline{\mathbf{R}}_h = \boxtimes_{a=1}^s \overline{R}_{h,a}. \quad (11)$$

Thus a full hyper in $R_1 \boxtimes R_2$ contains $R_1 \boxtimes R_2$ and $\overline{R}_1 \boxtimes \overline{R}_2$; it does not contain the two mixed tensor products unless they are separate matter entries.

4 Plethystic exponential and exact cutoffs

With flavor fugacities unrefined, define the representation-valued letter index

$$\mathcal{L}(t, y, u; g) = f_V(t, y, u) \sum_{a=1}^s \chi_{\mathfrak{ad}_a}(g_a) + f_H(t, y, u) \sum_h \hat{\chi}_h(g). \quad (12)$$

The free-field index before gauge projection is the plethystic exponential [1, 2]

$$\mathcal{Z}(t, y, u; g) = \text{PE}[\mathcal{L}] = \exp E(t, y, u; g), \quad (13)$$

$$E(t, y, u; g) = \sum_{j=1}^{\infty} \frac{1}{j} \mathcal{L}(t^j, y^j, u^j; g^j). \quad (14)$$

Since every letter begins at t^2 , a result through t^D needs only

$$1 \leq j \leq M = \left\lfloor \frac{D}{2} \right\rfloor. \quad (15)$$

The same lower bound implies that only $E^k/k!$ with $k \leq M$ can contribute. Consequently both the Adams sum and the ordinary exponential have finite, exact cutoffs at the requested order.

5 How FORM represents and expands the index

5.1 Formal character symbols

Each distinct pair “gauge-factor position, Dynkin labels” receives a commuting FORM function $C_p(j)$. Mathematically it denotes the j -th Adams operation

$$C_p(j) = \psi^j(\chi_{R_p}), \quad (\psi^j \chi)(g) = \chi(g^j). \quad (16)$$

For a bifundamental, FORM therefore sees $C_p(j)C_q(j)$. It expands only the coefficients and products of these formal objects; their Lie-theoretic decomposition is postponed until after the series is complete.

At Adams degree j , `_build_form_program` constructs

$$\mathcal{L}_j = f_V(t^j, y^j, u^j) \sum_{p \in \mathcal{V}} C_p(j) + f_H(t^j, y^j, u^j) \sum_m N_m \prod_{p \in m} C_p(j), \quad (17)$$

where $\mathcal{V}$ contains the adjoint character of every gauge factor and the matter monomials already include the full-hyper conjugate term from Eq. (10).

5.2 The generated FORM program in plain language

The central generated block has the following form:

Listing 1: Schematic FORM program generated by Python.

```
S m,n,y,idx1,idx2,j,z,u,t(:D);
CF d,C0,C1,...;
PolyRatFun d;
Function Kvec,Khyp;

L J=sum_(idx1,0,D/3,m^idx1)
    *sum_(idx2,0,D/3,n^idx2);
id m=t^3*y;
id n=t^3/y;
.sort

L itotal=sum_(j,1,D/2,(vector terms + hyper terms)/j);
id Kvec(t?,y?,u?)=J*(t^2*u^2-t^4/u^2-t^3*y-t^3/y+2*t^6);
id Khyp(t?,y?,u?)=J*(t^2/u-t^4*u);
.sort
```

FORM construct	Meaning in this calculation
S ... t(:D);	Declares commuting symbols. The maximum-power declaration automatically discards $t^{D+1}, t^{D+2}, \dots$ during term normalization [4, Sec. 7.143].
CF d,C0,...;	Declares commuting functions. The $C_p(j)$ objects are formal Adams-operated characters.
PolyRatFun d;	Makes <code>d(numerator,denominator)</code> the exact rational coefficient container, so equal monomials combine without floating point arithmetic [4, Sec. 7.114].
L name=...;	Creates a local FORM expression. <code>J</code> , <code>itotal</code> , <code>I</code> , and <code>result</code> are successive exact expressions.
sum_(j,a,b,X)	Generates the finite sum $\sum_{j=a}^b X$; the last argument is the summand [4, Sec. 8.67].
id lhs=rhs;	Applies an algebraic substitution. A question mark denotes a wildcard, so <code>Kvec(t?,y?,u?)</code> matches arguments such as <code>Kvec(t^j,y^j,u^j)</code> [4, Secs. 7.70–7.71].
.sort	Ends a FORM module, expands products, combines identical terms, and enforces the t cutoff before the next module [4, Sec. 2.3].

5.3 Why the auxiliary symbol z gives the exponential

Let $E = \text{itotal}$ and $M = \lfloor D/2 \rfloor$. The code begins with $I = zE$. For $k = 2, \dots, M$ it applies

$$z \mapsto 1 + \frac{zE}{k}. \quad (18)$$